

Sistemi Operativi

*I seguenti appunti sono a cura di **Catalin Ceban** e si basano sul corso di "Sistemi Operativi" erogato nell'A.A. 2026 dal professore Gabriele Tolomei per il C.d.L Informatica (33503)*

26 settembre 2026

Indice

1	Introduzione al corso	3
1.1	Lo spettro Hardware-Software	4
1.2	L'architettura di von Neumann (1945)	5
1.2.1	Il Bus di Sistema	7
1.2.2	Il ciclo di istruzione	7
1.2.3	L'Unità Centrale di Elaborazione (CPU)	8
1.2.4	Registri della CPU	8
1.3	Pipelining delle istruzioni	9
1.3.1	Hazard della pipeline	10
1.4	Instruction Set Architectures (ISA)	11
1.5	Processori Multi-Core	11
1.5.1	Prestazioni della CPU: Velocità di Clock e Cicli	11
1.6	La gerarchia di memoria	13
1.6.1	Il principio di località	14
1.6.2	Architettura della Cache Memory	14
1.6.3	Politiche di scrittura della Cache	15
1.6.4	Politiche di mapping della Cache Line	15
1.7	Il sistema Input/Output (I/O)	16
1.7.1	Protocolli di trasferimento I/O	17
1.8	La Interrupt Vector Table (IVT)	18
1.9	La Interrupt Service Routine (ISR)	19
1.10	Overhead degli Interrupt	19
1.11	Direct Memory Access (DMA)	20

Capitolo 1

Introduzione al corso

Per comprendere appieno il ruolo dei sistemi operativi, è utile richiamare una celebre affermazione del matematico e informatico Richard Hamming:

«*Lo scopo del calcolo è la comprensione, non i numeri.*» — Richard Hamming

In modo del tutto analogo, possiamo affermare che lo scopo primario di un **Sistema Operativo** è l'**astrazione**, e non la semplice gestione meccanica dell'hardware.

Tuttavia, per poter costruire astrazioni corrette, stabili ed efficaci, è indispensabile padroneggiare in primo luogo il funzionamento della macchina fisica sottostante.

Osservazione 1.0.1: Virtualizzazione dell'hardware

I buoni sistemi operativi si distinguono proprio per la loro capacità di **virtualizzare** con successo i diversi aspetti dell'architettura hardware, fornendo ai programmi applicativi un'interfaccia semplificata e indipendente dai dettagli implementativi del livello fisico.

In questo capitolo introduttivo verranno dunque fornite delle basi riguardanti l'architettura degli elaboratori, in quanto queste conoscenze sono essenziali e necessarie al fine di comprendere *come* e *dove* lavora il Sistema Operativo. In particolare verranno trattate:

1. **L'architettura di von Neumann:** la struttura e il layout generale dell'elaboratore.
2. **La CPU:** l'organizzazione interna comprendente registri, ALU, pipeline e suddivisione in core.
3. **La gerarchia di memoria:** le tecnologie SRAM/DRAM, il principio di località e la gestione delle memorie cache.
4. **Il sistema di Input/Output (I/O):** il ruolo dei controller, la gestione degli interrupt e il meccanismo DMA (*Direct Memory Access*).

1.1 Lo spettro Hardware-Software

Per comprendere il ruolo di un sistema operativo, è fondamentale analizzare la sua collocazione all'interno della gerarchia che separa il software applicativo dall'hardware sottostante. Ci si chiede quindi: *dove si posiziona esattamente il sistema operativo?*

L'architettura del sistema può essere suddivisa in tre macro-aree di esecuzione: lo **User Space** (spazio utente), il **Kernel Space** (spazio kernel) e la **Physical Machine** (macchina fisica).

Livello dell'Architettura	Ambito di Esecuzione
Applicazioni (<i>Python, Java, C++, Web</i>)	User Space
Librerie di Sistema (<i>libc</i>)	User Space
Sistema Operativo (<i>Kernel</i>)	Kernel Space
Firmware / Device Driver	Hardware / Low-Level Space
Hardware (<i>CPU, RAM, Controller I/O</i>)	Physical Machine

Parleremo in modo più approfondito del kernel e delle componenti di un Sistema Operativo più avanti nel corso. Come possiamo però vedere, il sistema operativo agisce come un vero e proprio *ponte* tra il livello logico (le applicazioni in esecuzione) e il livello fisico (gli effettivi componenti hardware).

Le astrazioni software fornite dai sistemi operativi devono necessariamente rispettare i limiti fisici imposti dall'hardware sottostante. La progettazione del software di sistema non può prescindere da questi vincoli, tra i quali spiccano in particolare:

Divario di velocità (*Speed gap*): Le CPU operano a frequenze molto elevate, riuscendo ad eseguire le istruzioni in frazioni di nanosecondo. Al contrario, i tempi di accesso alla memoria principale (DRAM) si misurano nell'ordine delle decine di nanosecondi. Questa marcata differenza prestazionale rende la memoria un frequente collo di bottiglia durante l'esecuzione dei processi.

Compromesso sulla memorizzazione (*Storage trade-off*): Esiste un vincolo fisico ed economico inscindibile tra velocità, capacità e costo della memoria. I dispositivi di storage più veloci (come le cache) sono estremamente costosi e di piccole dimensioni, mentre le memorie più economiche (come i dischi secondari) garantiscono capacità enormi ma a fronte di velocità decisamente inferiori.

Concorrenza (*Concurrency*): L'esecuzione parallela nei reali sistemi multi-core richiede un supporto hardware esplicito e dedicato per la sincronizzazione (ad esempio tramite istruzioni atomiche), indispensabile per evitare race condition e garantire la coerenza dei dati.

Osservazione 1.1.1: Ruolo del Sistema Operativo

Uno degli scopi primari nella progettazione degli algoritmi di un sistema operativo è proprio quello di mascherare ed erodere questi limiti fisici, offrendo ai programmi un'astrazione dell'hardware efficiente, omogenea e trasparente.

1.2 L'architettura di von Neumann (1945)

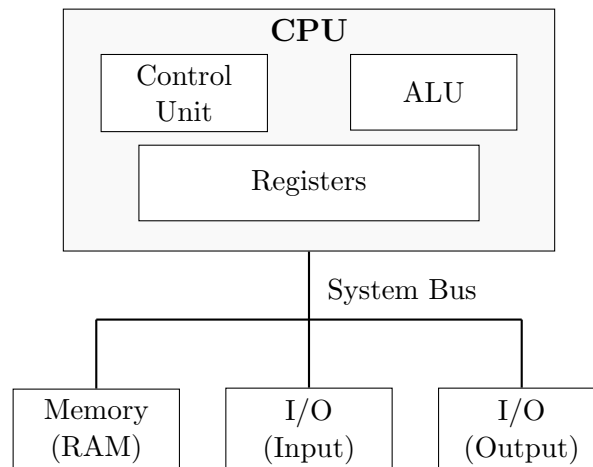
L'architettura di von Neumann rappresenta il fondamento teorico di quasi tutti i moderni calcolatori d'uso generale (*general-purpose*).

Tale modello si basa su due concetti chiave:

Concetto di programma memorizzato (*Stored-Program*): Le istruzioni e i dati condividono lo stesso spazio di memoria fisica (memorizzati in una memoria detta *Memoria Centrale*).

Esecuzione sequenziale: Le istruzioni vengono prelevate ed estratte una alla volta dalla memoria ed eseguite sequenzialmente dalla CPU. Non è previsto alcun tipo di concorrenza da questa architettura.

Di seguito forniamo uno schema grafico rappresentante la struttura del modello:



L'architettura si articola nei seguenti elementi funzionali:

Unità Centrale di Elaborazione (CPU): È il «cervello» del calcolatore, responsabile del coordinamento e dell'esecuzione di tutte le istruzioni. Al suo interno comprende:

- **Control Unit (Unità di Controllo):** Preleva le istruzioni dalla memoria, le decodifica e genera i segnali di controllo necessari al loro completamento.
- **ALU (Arithmetic Logic Unit):** Esegue le operazioni aritmetiche di base e le operazioni logiche elementari sui dati.
- **Registers (Registri):** Celle di memoria ad altissima velocità situate all'interno della CPU, impiegate per contenere temporaneamente dati, indirizzi di memoria o stati di esecuzione. Tra questi registri abbiamo, ad esempio:
 - **Program Counter (PC),** che si occupa di memorizzare l'indirizzo della prossima istruzione che deve essere eseguita dalla CPU.
 - **Instruction Register (IR),** che contiene il codice binario dell'effettiva istruzione puntata dal Program Counter.

Bus di Sistema (*System Bus*): È la struttura di collegamento condivisa (costituita da linee elettriche) attraverso la quale la CPU scambia indirizzi, dati e segnali di controllo con la memoria e le periferiche.

Memoria Principale (RAM): Contiene sia il codice dei programmi in esecuzione sia i dati necessari al funzionamento delle applicazioni. Nel modello di von Neumann, istruzioni e dati condividono lo stesso spazio di memoria ed i medesimi bus.

Interfacce di I/O (Input/Output): Moduli necessari a permettere l'interazione tra l'elaboratore e il mondo esterno, distinti in dispositivi di ingresso (*Input*, come tastiera e mouse) e di uscita (*Output*, come monitor e stampanti).

Tuttavia, come visto in precedenza, il principale svantaggio dell'architettura di von Neumann è proprio il fatto che le istruzioni e i dati condividono lo stesso spazio di memoria. Da ciò si origina il cosiddetto *von Neumann Bottleneck*, cioè un effettivo limite fisico in quanto una memoria non può essere acceduta da più dispositivi contemporaneamente. Un'alternativa a questo modello è infatti il cosiddetto **Modello Harvard**, che prevede spazi di memoria e bus fisicamente separati per istruzioni e dati.

1.2.1 Il Bus di Sistema

Entriamo più nel dettaglio riguardo ai bus di sistema, che hanno l'importante compito di traspostare qualsiasi tipo di dato tra una componente e l'altra.

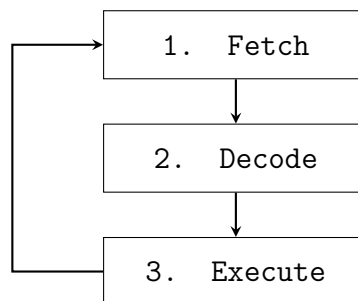
Bus Indirizzi (*Address Bus*): trasporta gli indirizzi di memoria dalla CPU verso i componenti di destinazione. Questo bus è **unidirezionale**, in quanto solo la CPU deve poter inviare l'indirizzo di memoria su cui lavorare. Inoltre, l'Address Bus ha una proprietà detta **Bus Width** (che fisicamente rappresenta quanti fili elettrici ci sono per quel bus) e che determina qual è il massimo numero di locazioni indirizzabili. La formula è molto semplice: se abbiamo $\text{width} = N$, il numero di locazioni indirizzabili è pari a 2^N .

Bus Dati (*Data Bus*): gestisce il trasferimento dei byte di dati effettivi tra i vari componenti. Rappresenta il canale sul quale viaggia fisicamente l'istruzione da eseguire oppure il dato da elaborare. Poiché i dati devono poter essere scritti e letti, la trasmissione lungo il Data Bus è di tipo *bidirezionale*. Anche questo bus possiede la proprietà di **Bus Width**, con significato analogo a quella dell'Address Bus, e questo valore in genere coincide con la Word Size del processore (dunque più è ampio il bus, maggiore è la quantità di dati che possono essere trasmessi simultaneamente).

Bus di Controllo (*Control Bus*): veicola i segnali di comando e di sincronizzazione del sistema, come le operazioni di lettura (*read*), scrittura (*write*), segnali di *reset* o richieste di *interrupt*. In sostanza, è il mezzo tramite cui la Control Unit (CU) definisce cosa fare con il dato e l'indirizzo selezionati.

1.2.2 Il ciclo di istruzione

Il **ciclo di istruzione** (o *fetch-decode-execute cycle*) rappresenta il processo fondamentale alla base del funzionamento di qualsiasi elaboratore. Ogni programma esegue questo ciclo in maniera continua.



Il ciclo si compone di tre fasi sequenziali gestite direttamente dalla CPU:

1. **Fetch (Prelevamento):** il contenuto del PC viene trasferito sull'Address Bus per poi prelevare l'effettiva istruzione puntata da quell'indirizzo. Alla fine di questa fase il valore del PC viene incrementato in modo di far passare all'istruzione successiva.
2. **Decode (Decodifica):** il processore analizza e interpreta il significato della sequenza di bit che compone l'istruzione. Questa operazione viene svolta in base alla specifica codifica scelta dall'architettura dell'elaboratore (ossia l'*Instruction Set Architecture* o ISA).
3. **Execute (Esecuzione):** viene svolta l'operazione richiesta dall'istruzione. Questa fase può comportare un calcolo eseguito dall'Unità Aritmetico-Logica (ALU) oppure un'operazione di lettura/scrittura in memoria.

Al termine della fase di esecuzione, il ciclo riparte immediatamente dalla fase di *fetch* per l'istruzione successiva.

1.2.3 L'Unità Centrale di Elaborazione (CPU)

La **Central Processing Unit (CPU)** rappresenta il "cervello" del calcolatore, essendo l'organo fondamentale preposto all'esecuzione delle istruzioni memorizzate in memoria.

L'architettura interna della CPU si articola su tre elementi costruttivi principali:

1. **Datapath (Unità Dati):** include l'**Arithmetic Logic Unit (ALU)**, preposti alla manipolazione e al calcolo effettivo sui dati e i registri General-Purpose (adibiti al contenimento di risultati intermedi per le operazioni della ALU).
2. **Control Unit (CU):** opera come coordinatore dell'intero microprocessore. Il suo compito è prelevare le istruzioni dalla memoria (*fetch*), decodificarle e pilotare opportunamente il datapath per consentirne l'esecuzione.
3. **Bus Interno:** insieme di canali e linee di interconnessione interne che mettono in comunicazione diretta i registri, l'ALU e la Control Unit.

1.2.4 Registri della CPU

Vediamo ora una carrellata veloce dei registri di sistema e delle loro funzionalità:

1. **Registri visibili all'utente (General-Purpose):** sono direttamente indirizzabili dalle istruzioni in linguaggio assembly per la manipolazione dei dati e l'esecuzione delle operazioni.

2. **Registri di controllo e di stato (Special-Purpose):** sono utilizzati dalla *Control Unit* (CU) per gestire lo stato di esecuzione del sistema e del processore. Tra i più rilevanti rientrano:

- **Program Counter (PC):** mantiene l'indirizzo della prossima istruzione da eseguire.
- **Instruction Register (IR):** contiene l'istruzione attualmente in fase di decodifica o esecuzione.
- **Processor Status Register (PSR):** memorizza le informazioni sullo stato corrente del processore (flag di condizione, livello di privilegio, ecc.).
- **Stack Pointer (SP):** contiene l'indirizzo di memoria della cima dello stack delle chiamate (ad esempio di chiamate delle funzioni).
- **Memory Address Register (MAR):** contiene l'indirizzo che sta per essere utilizzato dalla RAM. E' direttamente collegato all'Address Bus.
- **Memory Data Register (MDR):** contiene i byte di dati letti dalla RAM, ed è direttamente collegato al Data Bus.

1.3 Pipelining delle istruzioni

Nei processori più semplici, il ciclo di esecuzione delle istruzioni segue un approccio strettamente sequenziale. Come previsto dal modello classico dell'architettura di Von Neumann, il processo avviene in modo sequenziale: la CPU deve completare interamente le fasi di prelievo (*fetch*), decodifica (*decode*) ed esecuzione (*execute*) di una singola istruzione prima di poter passare a quella successiva. Questo schema, pur essendo semplice, comporta un notevole spreco di risorse hardware, poiché molte unità funzionali della CPU rimangono inattive durante le fasi in cui non sono direttamente coinvolte.

Per superare questa limitazione e migliorare le prestazioni del sistema, si introduce una tecnica di ottimizzazione fondamentale:

Definizione 1.3.1: Pipelining

Il **pipelining** è una tecnica di ottimizzazione hardware che consente di sovrapporre l'esecuzione di più istruzioni nel tempo, parallelizzando le diverse fasi del loro ciclo di vita.

Tale tecnica, almeno per le architetture di tipo RISC, si articola in 5 stadi:

1. **IF (Instruction Fetch):** prelievo dell'istruzione dalla cache di primo livello (L1 Cache) o dalla memoria principale (RAM).

2. **ID (Instruction Decode)**: decodifica dell'istruzione appena prelevata e lettura dei registri sorgente.
3. **EX (Execute)**: esecuzione del calcolo tramite l'unità aritmetico-logica (ALU) o calcolo dell'indirizzo di memoria.
4. **MEM (Memory Access)**: accesso alla memoria dati, che comporta un'operazione di lettura o scrittura verso la cache dati.
5. **WB (Writeback)**: scrittura del risultato finale all'interno del registro di destinazione.

Grazie a questa suddivisione, ogni stadio opera su un'istruzione differente in modo simultaneo durante ciascun ciclo di clock, consentendo un'elevata sovrapposizione delle operazioni e migliorando l'efficienza del processore.

1.3.1 Hazard della pipeline

In condizioni ideali, le architetture con pipeline consentono di raggiungere il massimo incremento prestazionale. Tuttavia, nella realtà possono verificarsi delle situazioni anomale note come **hazard** (o criticità), che impediscono all'istruzione successiva di essere eseguita nel ciclo di clock prestabilito.

Tali criticità si dividono principalmente in tre grandi categorie:

Hazard strutturali (*Structural Hazards*): Si verificano in presenza di un conflitto di risorse, ovvero quando due o più stadi della pipeline richiedono contemporaneamente l'accesso allo stesso componente hardware fisico.

Hazard sui dati (*Data Hazards*): Si manifestano quando un'istruzione dipende strettamente dal risultato di un'istruzione precedente che non ha ancora completato il suo percorso all'interno della pipeline.

Hazard di controllo (*Control Hazards*): Sono causati dalle istruzioni di salto condizionato o incondizionato (*branch*), le quali modificano il valore del *Program Counter* (PC). Ciò rende invalide le istruzioni che sono già state caricate (*fetched*) preventivamente nella pipeline.

Per risolvere tali criticità, le CPU moderne impiegano dei predittori di salto (**branch predictors**) per indovinare l'esito di un'istruzione di salto prima che essa venga effettivamente completata. Tuttavia, tali metodologie non verranno trattate.

1.4 Instruction Set Architectures (ISA)

Nel mondo dell'informatica sono stabiliti due standard principali riguardo al linguaggio adottato dalle istruzioni macchina:

CISC (Complex Instruction Set Computer): ogni istruzione macchina può comprenderne altre (sono infatti a lunghezza variabile), in modo da "semplificare" la scrittura del software per gli utenti.

RISC (Reduced Instruction Set Computer): mantiene solo le istruzioni principali del codice macchina (tutte hanno la stessa lunghezza).

1.5 Processori Multi-Core

Gli elaboratori di oggi aggirano i limiti di potenza e velocità dei processori collocando più processori (detti appunto *core*) su un singolo macro-processore. La caratteristica principale è proprio che ogni *core* è come se fosse "autonomo", in quanto possiede i suoi registri e le sue unità di controllo. A questo punto deve essere il Sistema Operativo a saper gestire l'architettura Multi-Core: deve saper far comunicare efficientemente i vari core per la condivisione di risorse senza che si verifichino problemi di sincronizzazione delle variabili. Discuteremo di queste tecniche quando parleremo di *concorrenza* e *multi-threading*.

1.5.1 Prestazioni della CPU: Velocità di Clock e Cicli

Il funzionamento della CPU è sincronizzato da un **clock di sistema** (*SystemClock*) oscillante, che scandisce il ritmo delle operazioni eseguite dal processore. Per comprendere le prestazioni di una CPU, è fondamentale analizzare due grandezze fondamentali legate a questo segnale:

- **Periodo di clock (T):** rappresenta la durata temporale di un singolo ciclo di clock (ad esempio 0.25 nanosecondi).
- **Frequenza di clock (f):** rappresenta il numero di cicli di clock al secondo, misurata in Hertz (Hz).

Queste due grandezze sono inversamente proporzionali tra loro e legate dalla relazione:

$$f = \frac{1}{T}$$

A titolo d'esempio, una CPU a 4.0 GHz è in grado di completare 4×10^9 cicli di clock in un solo secondo.

Tuttavia, non tutte le istruzioni macchina richiedono lo stesso tempo per essere eseguite: le istruzioni più semplici possono essere completate in un solo ciclo di clock, mentre le operazioni più complesse (come la divisione tra numeri in virgola mobile o gli accessi alla memoria principale) richiedono un numero elevato di cicli per poter essere portate a termine.

Per valutare e confrontare l'efficienza dei processori, risulta fondamentale definire delle metriche standard. Una delle domande chiave nell'architettura dei calcolatori è: quante istruzioni riesce ad eseguire una CPU per ogni ciclo di clock? Per rispondere a questo interrogativo, introduciamo due parametri fondamentali:

- **CPI (Cycles Per Instruction):** rappresenta il numero medio di cicli di clock necessari per eseguire una singola istruzione.
- **IPC (Instructions Per Cycle):** rappresenta l'inverso del CPI, ovvero il numero di istruzioni eseguite in un singolo ciclo di clock. La relazione matematica è la seguente:

$$\text{IPC} = \frac{1}{\text{CPI}}$$

Per calcolare il tempo totale necessario per eseguire un determinato programma, si utilizza la formula fondamentale delle prestazioni della CPU, che mette in relazione il numero di istruzioni, l'efficienza architetturale e la frequenza di clock:

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

Dove con *CPU Time* si intende il tempo totale necessario per eseguire tutte le istruzioni del programma. Analizzando i singoli componenti della formula:

- **Instruction Count:** il numero totale di istruzioni eseguite dal programma.
- **CPI:** il numero medio di cicli per istruzione.
- **Clock Cycle Time:** la durata di un singolo ciclo di clock (l'inverso della frequenza di clock).

1.6 La gerarchia di memoria

La velocità dei processori, nel corso degli ultimi anni, è cresciuta molto più velocemente (si potrebbe dire in maniera quasi esponenziale) rispetto alla velocità di accesso alle memorie RAM. Per continuare a velocizzare l'accesso alle risorse, gli architetti dei computer fanno riferimento ad una *gerarchia delle memorie*, una struttura piramidale che organizza le memorie dall'alto verso il basso, dove quella più in **cima** sono i **registri**, memorie *estremamente veloci*, situate all'interno della CPU, ma altrettanto costose; alla base di questa piramide si collocano i dispositivi di archiviazione di massa, come *SSD* e *HDD*, che sono più economiche ma decisamente più lente. Dunque, il compito di un bravo architetto di computer è quello di trovare il giusto compromesso a livello di prezzo e velocità, e il compito di un buon sistema operativo è quello di saper sfruttare queste risorse nel modo più efficiente possibile.

Possiamo riassumere il comportamento della struttura con una regola fondamentale: **scendendo nella gerarchia, la velocità di accesso diminuisce, la capacità aumenta e il costo per byte diminuisce.**

Livello	Componente	Capacità tipica	Latenza di accesso
L0	CPU Registers	< 1 KiB	< 0.5 ns
L1	L1 Cache (per core)	32 – 64 KiB	1 – 2 ns
L2	L2 Cache (per core)	256 – 512 KiB	3 – 5 ns
L3	L3 Cache (shared)	8 – 64 MiB	10 – 20 ns
L4	Main Memory (DRAM)	8 – 64 GiB	60 – 100 ns
L5	SSD (NAND Flash)	256 GiB - 2 TiB	10 – 100 μ s
L6	HDD (Magnetic Disk)	1 – 10 TiB	5 – 10 ms

La memoria principale è strutturata come una griglia monodimensionale di indirizzi di byte (*byte addresses*). Ogni cella possiede un indirizzo univoco a cui corrisponde un determinato contenuto memorizzato.

Indirizzo fisico	Contenuto
0x00000000	01101010
0x00000001	11001100
...	...
0xFFFFFFFF	00000000

1.6.1 Il principio di località

L'efficacia della gerarchia di memoria (*memory hierarchy*) si basa sul "principio di località", che definisce una serie di principi secondo i quali lavora la CPU per ottimizzare l'accesso alle informazioni. I programmi in esecuzione non accedono infatti alla memoria in modo casuale, ma presentano due forme fondamentali di località:

Località temporale (*Temporal Locality*): se una determinata locazione di memoria viene acceduta una volta, è molto probabile che la stessa locazione venga acceduta nuovamente nel futuro prossimo. Ad esempio, in un ciclo **for** in cui si accede alla variabile **i** per ogni iterazione del ciclo, la CPU dovrà accedere sempre alle stesse variabili e dunque terrà memorizzato in qualche registro vicino alla CPU l'indirizzo di memoria di quelle variabili.

Località spaziale (*Spatial Locality*): se viene acceduta una specifica locazione di memoria, è molto probabile che anche le locazioni ad essa vicine o adiacenti vengano accedute nel prossimo futuro. Tenendo sempre l'esempio del ciclo **for**, vedendo che tali locazioni di memoria vengono usate spesso, la CPU tende a memorizzare quei valori nei registri più vicini ad essa, in modo da renderne l'accesso quasi immediato.

1.6.2 Architettura della Cache Memory

Le memorie cache non gestiscono i dati a livello di singoli byte, ma operano su blocchi di dimensione fissa denominati **cache line** (la cui dimensione tipica è di 64 byte).

Ogni entry all'interno di una cache line è composta da tre elementi fondamentali:

Valid Bit: Un bit di controllo che indica se i dati presenti nella linea di cache sono validi ed aggiornati. Questo flag permette di verificare se il dato cercato è effettivamente presente ed utilizzabile all'interno della cache.

Tag Bit: Identifica l'indirizzo di memoria originale (in RAM) da cui sono stati reperiti i dati. Agisce in modo analogo alla chiave di un dizionario: permette di stabilire in modo univoco a quale posizione della memoria principale appartenga il blocco.

Data Block: Contiene l'effettivo blocco di dati copiato dalla memoria RAM (tipicamente 64 byte), ovvero il contenuto informativo richiesto situato a quell'indirizzo.

Nel momento in cui la CPU richiede un determinato indirizzo di memoria (*memory address*), si possono verificare due situazioni distinte:

Cache Hit (Dato trovato) Si ha un *cache hit* quando la CPU individua con successo l'indirizzo e il dato cercato direttamente all'interno della memoria cache.

Cache Miss (Dato non trovato) Si ha un *cache miss* quando la CPU non trova l'indirizzo richiesto all'interno della memoria cache, rendendo necessario il recupero del dato da memorie più lente.

1.6.3 Politiche di scrittura della Cache

Nel momento in cui la CPU scrive i dati in un cache block, possono essere applicate due politiche per mantenere aggiornata la RAM:

Write-Through: la CPU scrive contemporaneamente sia nella cache che nella memoria centrale. Il vantaggio è ovviamente una sincronizzazione costante dei dati; lo svantaggio è che una semplice operazione di scrittura di un dato sarà sempre lenta in quanto si dovrà riportare anche in memoria centrale, e dunque elimina il vantaggio di utilizzo della cache.

Write-Back: la CPU scrive solo nella cache line e la marca come "modificata". L'aggiornamento effettivo nella memoria centrale si verifica solo quando tale cache line viene espulsa dalla cache per fare spazio ad altri dati. Vantaggioso in quanto viene mantenuta la velocità di memorizzazione di dati all'interno della cache, ma rimane il problema possibile della sincronizzazione.

1.6.4 Politiche di mapping della Cache Line

Una questione fondamentale nell'organizzazione delle architetture dei calcolatori riguarda il criterio secondo cui un blocco della memoria principale (*memory block*) viene posizionato all'interno di uno slot della memoria cache (*cache slot*).

Per gestire questa mappatura ed individuare la posizione specifica in cui collocare i dati, si adottano principalmente tre schemi di progettazione:

Direct Mapped (Mappatura Diretta): Ogni blocco di memoria principale viene mappato in modo univoco ed esatto in un solo ed unico *cache slot*. Vantaggioso in quanto siamo sicuri di dove ritrovare i blocchi di memoria della cache. Possono tuttavia verificarsi conflitti di indirizzi in caso di mappatura nella stessa locazione.

Fully Associative (Completamente Associativa): Ogni blocco di memoria principale può essere collocato in un qualsiasi *cache slot* libero all'interno della cache. Tale approccio elimina i problemi di conflitto di mappatura, ma potendo posizionare un blocco in qualsiasi punto, dovremo cercare in qualsiasi punto possibile.

Set Associative (Associativa a Insiemi): Rappresenta un compromesso tra le due politiche precedenti. La memoria cache viene suddivisa in un determinato numero di insiemi (*set*), ciascuno composto da un numero fisso di slot. Un blocco di memoria viene mappato in modo rigido a uno specifico *set*, ma all'interno di quel *set* può risiedere in un qualsiasi slot libero.

1.7 Il sistema Input/Output (I/O)

Affinché un calcolatore sia effettivamente utile ed utilizzabile, esso deve necessariamente comunicare con l'ambiente esterno mediante opportuni dispositivi. È importante sottolineare che i dispositivi di Input/Output non si limitano unicamente alle periferiche di interazione diretta con l'utente (come quelle adibite all'inserimento di testo o alla visualizzazione grafica), ma comprendono un'ampia varietà di categorie:

- **Dispositivi di memorizzazione secondaria (Storage Devices):** memorie di massa persistenti quali SSD, HDD e unità USB Flash.
- **Dispositivi di rete (Network Devices):** schede di rete (NIC), sia cablate (Ethernet) che wireless (Wi-Fi), per la trasmissione e ricezione di dati attraverso la rete.
- **Interfacce utente (User Interfaces):** periferiche standard di interazione uomo-macchina, tra cui tastiera, mouse e schermo.

Tutti i trasferimenti di dati da CPU a dispositivi di I/O sono gestiti da appositi sistemi hardware o software specializzati nel controllo di quel specifico dispositivo o di quella specifica categoria di dispositivi.

Device Controller (Hardware) Consiste in un effettivo microchip integrato sulla scheda madre o alloggiato su una scheda di espansione che controlla direttamente il dispositivo fisico.

Device Driver (Software) Si tratta di un modulo del sistema operativo che conosce la sintassi e la logica per comunicare correttamente con il dispositivo di I/O.

Una questione fondamentale nella gestione delle periferiche riguarda il meccanismo con cui il processore interagisce con l'hardware: *come legge e scrive la CPU nei registri del device controller?*

Per rispondere a questa domanda si distinguono principalmente due approcci architetturali:

1. **Port-Mapped I/O (PMIO):** Prevede l'utilizzo di **istruzioni CPU dedicate** (come ad esempio le istruzioni *IN* e *OUT* nelle architetture x86). Questo approccio nasce dal fatto che, a livello di *control bus*, le consuete operazioni di *LOAD* e *STORE* possono essere utilizzate solo se l'indirizzo di destinazione fa riferimento a una locazione di memoria principale. Si rende quindi necessaria la definizione di uno spazio di indirizzamento separato e di istruzioni specifiche riservate alla comunicazione con i dispositivi di I/O.
2. **Memory-Mapped I/O (MMIO):** Rappresenta la soluzione standard e moderna, che consiste nella predisposizione di uno spazio di memoria e dei registri interamente dedicati a questi dispositivi.

Un controller espone tipicamente tre classi fondamentali di registri:

Status register: registro di sola lettura, riporta lo stato corrente del dispositivo (BUSY, READY, ...).

Command register: sola scrittura, indica al dispositivo *cosa* fare.

Data register: lettura e scrittura, contiene i dati inviati/ricevuti al/dal dispositivo.

1.7.1 Protocolli di trasferimento I/O

Una delle problematiche fondamentali dell'architettura di un elaboratore riguarda la modalità con cui la CPU coordina e gestisce il trasferimento dei dati da e verso un dispositivo di Input/Output. Per rispondere a questa esigenza, si adottano principalmente tre protocolli:

1. **Programmed I/O (Polling)**: la CPU controlla ciclicamente lo stato del dispositivo per verificare se è pronto a trasferire i dati. Tale approccio può avere senso ed essere efficiente quando si devono trasferire pochissimi dati, in quanto evita l'overhead legato alla gestione delle interruzioni.
2. **Interrupt-Driven I/O**: come già accennato durante l'analisi del *control bus*, in questo schema è il dispositivo a notificare alla CPU l'avvenuto completamento di un'operazione tramite un segnale di interruzione. Risulta una modalità nettamente meno onerosa rispetto al polling quando si devono trasmettere quantitativi di dati più consistenti. In particolare, appena la CPU riceve questo segnale di interruzione, effettua una "fotografia" dello stato corrente dei registri e passa all'esecuzione di una particolare *routine* (**Interrupt Service Routine (ISR)**). Nello specifico, tutte le routine sono memorizzate all'interno di una struttura dati detta **Interrupt Vector Table** a cui la CPU ha accesso e che contiene i puntatori ad ogni specifica ISR.
3. **Direct Memory Access (DMA)**: un modulo hardware dedicato si occupa di trasferire i dati direttamente tra la memoria principale e il dispositivo di I/O, senza che la CPU debba intervenire per ogni singolo byte, riducendo drasticamente il carico di lavoro del processore.